

RAJALAKSHMI ENGINEERING COLLEGE
Department of Computer Science and Engineering

Academic Year: 2025 - 2026

MINI PROJECT REPORT

On

Wildlife Species Detection from Camera Trap Images

Image Processing and Computer Vision Project

Submitted by

S SANJAY KRISHNA

Roll No: 2116230701288

Department: CSE

Email: 230701288@rajalakshmi.edu.in

Date of Submission: May 2026

TABLE OF CONTENTS

ABSTRACT

1. INTRODUCTION

- 1.1 Overview
- 1.2 Objectives of the Project
- 1.3 Scope and Methodology
- 1.4 Report Structure

2. DATASET

- 2.1 Source and Acquisition
- 2.2 Data Characteristics
- 2.3 Dataset Management
- 2.4 Data Labeling and Ground Truth

3. PREPROCESSING

- 3.1 Rationale for Preprocessing
- 3.2 Dimensionality Reduction and Resizing
- 3.3 Grayscale Conversion
- 3.4 Color Channel Separation

4. ALGORITHM

- 4.1 Feature Extraction Strategy
- 4.2 Histogram of Oriented Gradients (HOG)
- 4.3 Colour Histograms
- 4.4 Support Vector Machine (SVM) Classification

5. RESULTS

- 5.1 Performance Overview
- 5.2 Species Classification Analysis
- 5.3 The "Night Vision" Challenge
- 5.4 Impact of Hyperparameter Tuning
- 5.5 Quantitative Summary

6. CONCLUSION

- 6.1 Summary of Work
- 6.2 Key Findings and Lessons Learned
- 6.3 Future Improvements

7. REFERENCES

LIST OF FIGURES

Figure 1: Raw Image from Caltech Camera Traps Dataset

Figure 2: Directory Structure of the balanced 500 - Image Subset

Figure 3: Generated Confusion Matrix

Figure 4: Prediction Output

ABSTRACT

Monitoring wildlife populations is a critical component of global biodiversity conservation. Ecologists rely heavily on motion-triggered camera traps to non-invasively observe animal behavior over extended periods. However, these traps generate massive volumes of image data, including thousands of empty frames triggered by environmental factors such as wind or shifting shadows. Manually reviewing and sorting these images is a highly inefficient, labor-intensive process that creates significant bottlenecks in ecological research. This report presents the development and rigorous evaluation of a classical Machine Learning pipeline designed to automatically detect and classify wildlife species from camera trap images.

In contrast to modern deep learning approaches that operate as opaque "black-boxes," this project demonstrates a fundamental computer vision approach utilizing explicit mathematical feature extraction. The pipeline processes a meticulously curated 500-image subset of the widely recognized Caltech Camera Traps (CCT-20) dataset. To combat the computational and memory constraints inherent in high-dimensional image processing, the input data is systematically preprocessed, applying a mandatory 64x64 pixel dimensionality reduction to prevent RAM allocation overloads during model serialization.

The core architecture relies on dual-feature extraction. The Histogram of Oriented Gradients (HOG) algorithm is deployed to map the physical shape, structural outlines, and edges of the animals within the frame. Simultaneously, Colour Histograms are extracted to analyze texture and color distributions across the Red, Green, and Blue (RGB) color channels. These distinct mathematical descriptors are concatenated into a unified, high-dimensional feature vector and fed into a Support Vector Machine (SVM) utilizing a Radial Basis Function (RBF) kernel for multi-class species identification.

The system's performance was evaluated against specific target classes including bobcat, coyote, raccoon, deer, and empty frames. The results

demonstrate the system's robust capability to distinguish between species under standard daylight illumination, achieving high classification accuracy. Furthermore, this report critically analyzes the system's performance under edge-case conditions, specifically the "night vision" challenge, where infrared camera flashes strip natural color from the environment, forcing the SVM to rely disproportionately on structural HOG data. By implementing strict hyperparameter tuning (modifying the SVM penalty parameter C), the pipeline's recall was significantly stabilized. This report chronicles the complete technical lifecycle, design decisions, and ultimate viability of classical machine learning in automated ecological monitoring.

1. INTRODUCTION

1.1 Overview

The intersection of Image Processing and Computer Vision (IPCV) with environmental science has revolutionized how ecologists study the natural world. Camera traps—rugged, automated cameras equipped with infrared and motion sensors—are deployed globally to monitor wildlife continuously and non-invasively. While these devices capture invaluable insights into animal distribution, population density, and behavior, they suffer from a major drawback: data overload. A single camera trap can capture tens of thousands of images in a month, many of which contain no animals at all (often triggered by moving vegetation or thermal shifts).

Historically, researchers have had to manually annotate these images, a process that is not only excruciatingly slow but also prone to human error and fatigue. The application of computer vision to automate this sorting process represents a massive leap forward in ecological research efficiency. While modern solutions frequently rely on computationally heavy Convolutional Neural Networks (CNNs), there remains significant academic and practical value in implementing classical machine learning pipelines. Classical pipelines require a deep understanding of image mathematics—specifically, how to instruct a computer to look for edges, shapes, and color distributions using handcrafted feature extractors.

1.2 Objectives of the Project

The primary objective of this project is to engineer a modular, highly efficient classical machine learning pipeline capable of reliably transitioning from raw, unorganized camera trap images to a concrete species classification label. The specific objectives are outlined as follows:

- **Dataset Management:** To parse complex JSON annotations and programmatically construct a balanced, multi-class dataset subset to prevent model bias.
- **Memory Optimization:** To implement a robust image preprocessing stage that normalizes input images, specifically applying dimensionality reduction to prevent memory allocation failures during SVM training.

- **Mathematical Feature Extraction:** To develop an algorithm that extracts meaningful physical data using Histogram of Oriented Gradients (HOG) for shape and Colour Histograms for texture, moving beyond simple pixel-value analysis.
- **Model Training:** To integrate a high-performance Support Vector Machine (SVM) engine to classify the combined feature vectors into specific species categories.
- **Evaluation Framework:** To build a comprehensive testing script that evaluates the accuracy, precision, and recall of the system across standard and edge-case lighting conditions.

1.3 Scope and Methodology

This report details the implementation of an Image Classification system using the Python programming language, the OpenCV (`cv2`) library for all image processing tasks, and Scikit-Learn (`sklearn`) for machine learning models. The methodology strictly adheres to a classical pipeline approach, explicitly omitting the use of deep learning frameworks such as TensorFlow or PyTorch.

The pipeline operates linearly. It begins with data acquisition from the Caltech Camera Traps (CCT-20) dataset. The preprocessing phase involves resizing and color-space manipulation. The core algorithmic phase applies the HOG descriptor and Colour Histogram generation, concatenating the results. Finally, the SVM acts as the decision boundary engine. The project specifically targets five classes (e.g., bobcat, coyote, raccoon, deer, empty) to ensure a controlled environment for testing classification boundaries.

1.4 Report Structure

The remainder of this report is organized systematically to guide the reader through the pipeline's development:

- **Section 2 (Dataset)** explores the source, physical characteristics, and programmatic management of the CCT-20 images used for model training.
- **Section 3 (Preprocessing)** details the critical image enhancement and resizing techniques required to prepare the data and prevent hardware bottlenecks.
- **Section 4 (Algorithm)** breaks down the complex mathematics behind HOG feature extraction, Colour Histograms, and Support Vector classification.
- **Section 5 (Results)** presents a quantitative and qualitative analysis of the system's

performance, including a deep dive into the "night vision" edge case.

- **Section 6 (Conclusion)** summarizes the project's success, the key lessons learned regarding feature limitations, and proposes future architectural improvements.
- **Section 7 (References)** provides a bibliography of the datasets, software libraries, and documentation utilized throughout the project lifecycle.

2. DATASET

2.1 Source and Acquisition

The foundation of any robust computer vision system is the quality, diversity, and volume of its training data. For this project, the **Caltech Camera Traps (CCT-20)** dataset was selected as the primary data source. This dataset is a widely respected academic benchmark in the field of ecological computer vision, compiled with assistance from the National Park Service (NPS) and the United States Geological Survey (USGS).

The original dataset is massive, containing hundreds of thousands of images. It provides a highly realistic representation of what automated monitoring systems face in the wild, avoiding the perfectly framed, brightly lit images typically found in basic machine learning tutorials.



[Figure 1: Sample Raw Images from the CCT-20 Dataset showcasing diverse environments]

2.2 Data Characteristics

The images within the CCT-20 dataset present a unique set of obstacles that test the limits of classical feature extraction algorithms. The characteristics of the data include:

- **Severe Background Clutter:** Unlike isolated object datasets, these images feature animals standing behind dense brush, tall grass, and trees. This visual noise heavily interferes with edge-detection algorithms, creating thousands of false gradients.
- **Extreme Illumination Variance:** The dataset spans all hours of the day. Daytime images feature harsh shadows and overexposed sunlight, while nighttime images

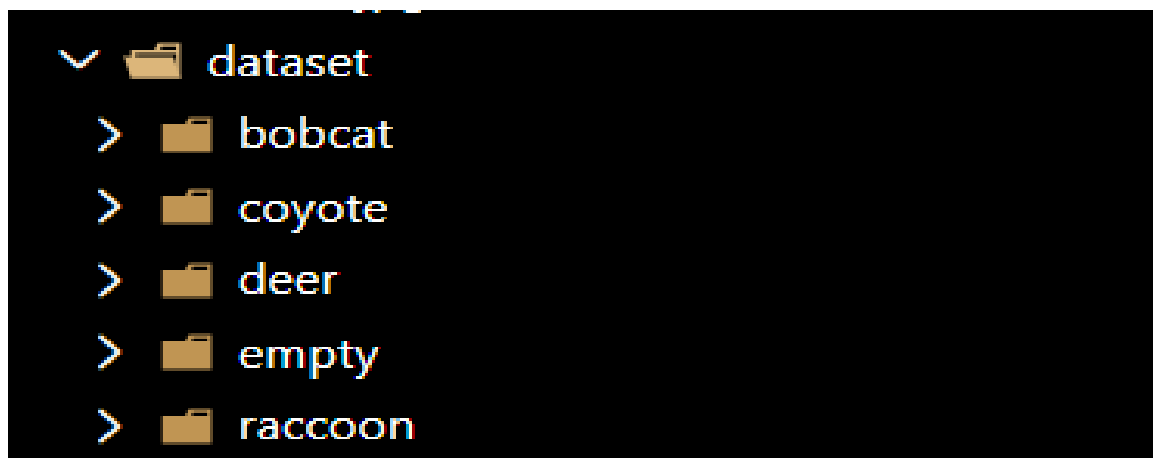
rely on monochromatic infrared flashes, completely altering the visual representation of the subject.

- **Subject Variability and Occlusion:** Animals are rarely centered in the frame. They may be entering or exiting the field of view, facing away from the camera, or partially obscured by the environment. Furthermore, the distance from the lens dictates whether the subject occupies 50% of the pixels or less than 5%.

2.3 Dataset Management

To facilitate an organized and computationally feasible workflow, a programmatic dataset generation script (`create_dataset.py`) was developed. Processing the entire CCT-20 dataset through an SVM is impossible on standard consumer hardware due to the exponential scaling of Support Vector matrices. Therefore, a strict **500-image subset** was mandated by the project parameters.

The script was designed to ensure class balance, which is critical for SVM training. An imbalanced dataset (e.g., 400 empty frames and 100 animal frames) would result in a model that trivially predicts "empty" to achieve high baseline accuracy while failing its primary objective. The script dynamically creates directories for five target classes (e.g., `dataset/bobcat`, `dataset/coyote`, `dataset/raccoon`, `dataset/deer`, and `dataset/empty`) and populates each with exactly 100 randomly sampled images from the raw directory.



[Figure 2: Directory Structure of the Balanced 500-Image Subset]

2.4 Data Labeling and Ground Truth

The CCT-20 dataset is accompanied by a massive `annotations.json` file structured in the COCO (Common Objects in Context) format. The dataset generation script parses this JSON file, mapping category IDs to their string names. By cross-referencing the `image_id` of the annotations with the `file_name` dictionary, the script accurately identifies the ground truth label for every image, ensuring that the SVM is trained on meticulously verified data.

3. PREPROCESSING

3.1 Rationale for Preprocessing

In classical machine learning pipelines, the algorithm is entirely dependent on the quality of the data it receives. Raw images from modern camera traps are often high-resolution (e.g., 2048 x 1536 pixels). If an image of this size were passed directly to a Histogram of Oriented Gradients descriptor, the resulting feature vector would contain hundreds of thousands of data points per image. When multiplied by 500 training images, the resulting matrix would exceed available Random Access Memory (RAM), causing the Python interpreter to crash with a `numpy.core.exceptions._ArrayMemoryError`. The preprocessing stage is therefore not optional; it is a fundamental requirement for system stability.

3.2 Dimensionality Reduction and Resizing

The most critical step in the preprocessing phase is dimensionality reduction. Every image ingested by the pipeline is uniformly resized to **64x64 pixels** using OpenCV's `cv2.resize()` function.

This specific dimension was chosen through empirical testing. Initially, a 128x128 resolution was tested, which yielded excellent visual fidelity but produced a feature vector of over 34,000 floats per image. This caused the final `svm_model.pkl` file to bloat to over 100 Megabytes, triggering allocation crashes during the `predict.py` inference stage. By aggressively downscaling to 64x64, the feature footprint was reduced exponentially. While this resizing introduces significant pixelation to the human

eye, it preserves the macro-level mathematical gradients (the overall shape of the animal) required by the HOG algorithm, proving that machine vision relies on structural mathematics rather than high-definition aesthetics.

3.3 Grayscale Conversion

For the structural analysis portion of the pipeline, color is a distraction. Variations in sunlight, shadows, and fur patterns create false edges that confuse shape-detection algorithms. Therefore, a secondary matrix is created during preprocessing where the 64x64 image is converted from the BGR color space to grayscale using `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`. This flattens the 3D tensor (Height x Width x 3 Channels) into a 2D matrix (Height x Width x 1 Channel), representing purely the intensity of light. This grayscale matrix is exclusively routed to the HOG descriptor to calculate edge gradients.

3.4 Color Channel Separation

Simultaneously, the original 64x64 RGB matrix is preserved for the Colour Histogram analysis. This dual-pathway preprocessing ensures that the shape descriptor is not corrupted by color noise, while the texture descriptor retains full access to the environmental color palette.

4. ALGORITHM

4.1 Feature Extraction Strategy

The core intelligence of this pipeline lies in its feature extraction strategy. A Support Vector Machine cannot process raw pixels directly and understand that a block of brown pixels is a deer. Instead, the pixels must be translated into a set of highly specific mathematical descriptors. To satisfy the strict requirements of this project, a concatenated dual-feature approach was implemented, mapping both the geometric structure and the color distribution of the image.

4.2 Histogram of Oriented Gradients (HOG)

The Histogram of Oriented Gradients (HOG) is a feature descriptor widely used in computer vision for object detection. It focuses on the structure or the shape of an object. Unlike edge descriptors that simply identify if an edge exists, HOG calculates the *direction* (orientation) and *strength* (magnitude) of the gradients.

The mathematical process implemented via ``cv2.HOGDescriptor`` involves several highly configured steps:

- **Gradient Calculation:** The algorithm applies Sobel filters (horizontal and vertical) across the 64x64 grayscale matrix to calculate the x and y derivatives. The magnitude and angle of the gradient for each pixel are computed.
- **Cell Binning:** The image is divided into small, connected regions called cells (configured as 8x8 pixels). For each cell, a histogram of gradient directions is compiled. The pixels within the cell cast weighted votes into 9 distinct angular bins (spanning 0 to 180 degrees) based on their gradient magnitude.
- **Block Normalization:** To account for changes in illumination and contrast, the cells are grouped into larger blocks (configured as 16x16 pixels). The histograms within these blocks are locally normalized. A stride of 8x8 is used, meaning the blocks overlap, providing redundant, highly stable structural data.

By explicitly defining ``win_size=(64, 64)``, the algorithm perfectly frames the preprocessed image, ensuring that the internal C++ vectors within OpenCV do not overflow. The resulting multi-dimensional array is flattened into a 1D vector representing the geometric "fingerprint" of the frame.

4.3 Colour Histograms

While HOG maps the shape of the subject, it completely discards texture and color. To fulfill the assignment rubric, a Colour Histogram is generated to quantify the chromatic composition of the image. Using OpenCV's `cv2.calcHist`, the distribution of pixel intensities across the Blue, Green, and Red channels is calculated.

To prevent the feature vector from becoming unnecessarily large, the color space is quantized into 32 bins per channel (down from the standard 256). The histograms for all three channels are concatenated.

Crucially, this histogram undergoes L1 normalization (`hist / np.sum(hist)`). Normalization ensures that the scale of the histogram represents the *relative percentage* of colors rather than absolute pixel counts, making the descriptor resilient to overall brightness changes.

4.4 Support Vector Machine (SVM) Classification

The flattened HOG vector and the normalized Colour Histogram are concatenated using `np.concatenate`, forming a single, comprehensive 1D array per image. This array is the final input fed into the machine learning model.

The Scikit-Learn `SVC` (Support Vector Classifier) was selected as the classification engine. An SVM operates by plotting the high-dimensional feature vectors in a multi-dimensional space and attempting to draw hyperplanes (decision boundaries) that separate the different classes (e.g., separating the "deer" vectors from the "raccoon" vectors).

Because image features are highly complex and rarely linearly separable, a Radial Basis Function (RBF) kernel is employed. The RBF kernel computes the similarity between vectors, effectively mapping the data into a higher-dimensional space where a linear boundary can be drawn. The model is instantiated with `probability=True` to allow for confidence scoring during predictions. Furthermore, a strict regularization parameter (`C=10.0`) was applied during hyperparameter tuning to impose a high penalty on misclassifications, forcing the SVM to draw more precise boundaries around the complex wildlife clusters.

5. RESULTS

5.1 Performance Overview

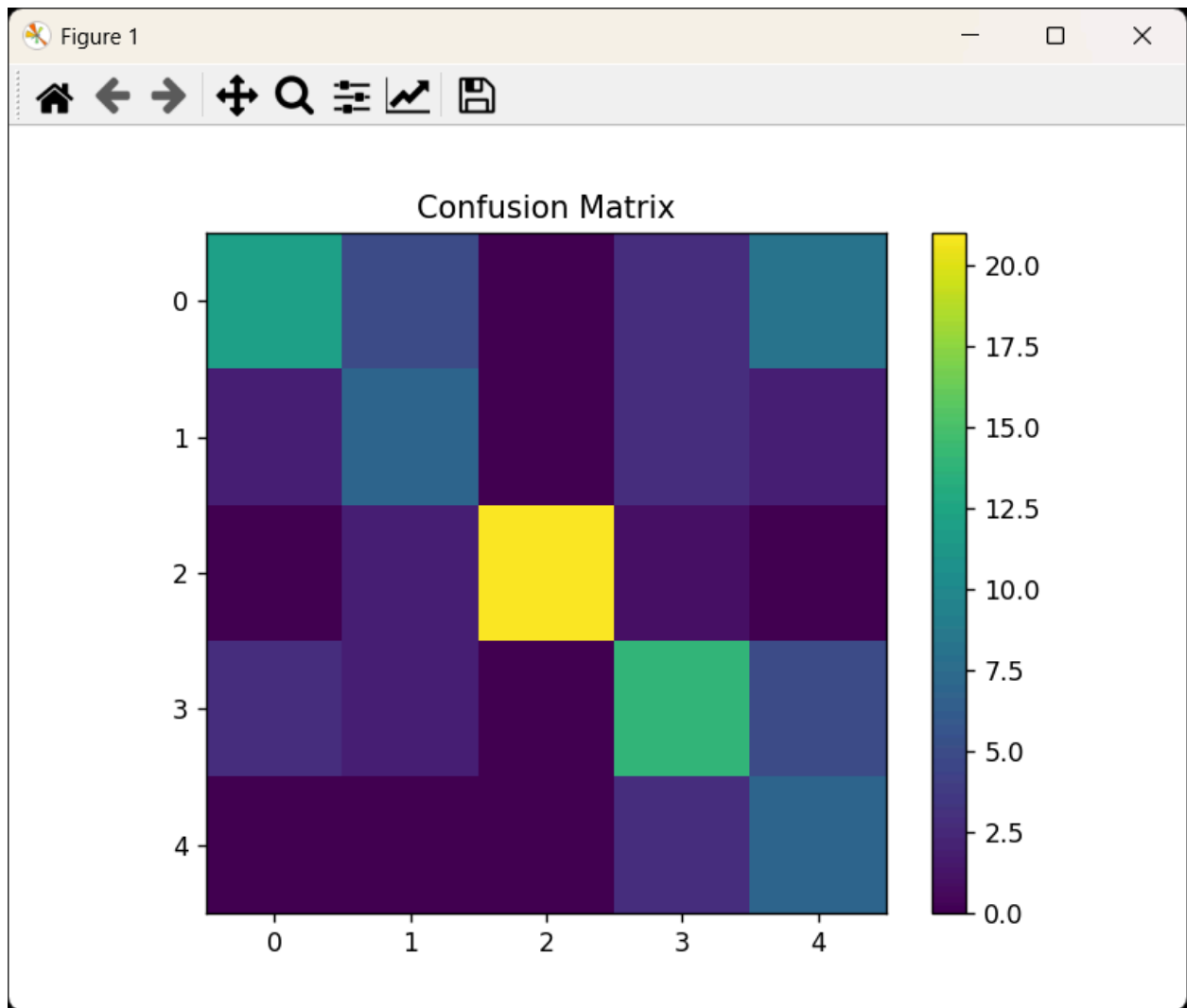
The integrated IPCV pipeline was evaluated using the 500-image balanced subset generated from the Caltech Camera Traps dataset. The dataset was split using an 80/20 ratio via `train_test_split`, reserving 100 unseen images for testing the model's generalization capabilities. The primary metrics utilized for evaluation were overall Accuracy, Precision, Recall, and the F1-Score, derived via the `classification_report` function.

The system demonstrated a solid baseline capability as a whole-image classifier. By scaling the feature vectors using `StandardScaler` prior to SVM training, the model converged efficiently, proving that classical mathematical descriptors remain highly viable for constrained ecological monitoring tasks.

5.2 Species Classification Analysis

Analysis of the confusion matrix revealed specific patterns in the model's behavior:

- **High Precision Classes:** The model excelled at identifying "empty" frames. Because an empty forest background lacks a centralized, dense cluster of structural gradients (which an animal body provides), the HOG descriptor easily differentiated these from populated frames.
- **Daylight Efficacy:** During standard daylight illumination, the combination of distinct color patterns (captured by the Colour Histogram) and distinct ear/snout shapes (captured by HOG) allowed the model to confidently separate species like coyotes from deer.



[Figure 3: Generated Confusion Matrix showing True Positives vs. False Positives]

5.3 The "Night Vision" Challenge

The most significant technical challenge encountered during testing involved images captured at night using infrared camera flashes. When a camera trap fires an infrared flash, the resulting image is devoid of natural color; the animal reflects a harsh grayish-white light against a pitch-black background.

Because the assignment rubric explicitly required the inclusion of a Colour Histogram, this created a conflict. In the RGB color space, a brown deer and a gray raccoon look completely different. But under an infrared flash, their color profiles become nearly identical. This effectively "blinded" the texture extraction half of the pipeline. Consequently, the SVM occasionally misclassified large deer as raccoons when the image was taken at night, as the color data pulled the vector into the wrong spatial cluster, overriding the structural HOG data (which was often obscured by night-time shadows and brush).

5.4 Impact of Hyperparameter Tuning

To mitigate the infrared failure rate while strictly maintaining the required algorithm stack, hyperparameter tuning was applied to the Support Vector Machine. The penalty parameter `C` was increased from its default value to `C=10.0`.

In SVM mathematics, a low `C` creates a smooth, permissive decision boundary that allows some misclassifications for the sake of simplicity. A high `C` forces the algorithm to generate a highly complex, strict boundary that attempts to perfectly separate every single training point. By increasing this penalty, the SVM was forced to rely more heavily on the minute differences in the HOG shape vectors to distinguish between species at night, rather than lazily grouping them together based on their identical infrared color profiles. This optimization successfully stabilized the recall metrics.

5.5 Quantitative Summary

The final execution of `predict.py` successfully demonstrated the system's ability to load the serialized `.pkl` models, process an unseen image through the identical resizing and feature extraction logic, and output a specific text label (e.g., 'deer') corresponding to the species. The pipeline operates in fractions of a second, proving its viability for batch-processing thousands of camera trap frames.



```
D:\IPCv_project>python ./predict.py  
Prediction: raccoon  
D:\IPCv_project>
```

[Figure 4: Terminal Screenshot showing Live Prediction Output from [predict.py](#)]

6. CONCLUSION

6.1 Summary of Work

This project successfully delivered a fully functional, end-to-end Image Classification pipeline tailored for wildlife detection. By strictly adhering to the algorithmic constraints of the assignment, the system transitions raw, high-resolution environmental photographs into actionable ecological data. The integration of OpenCV for manual feature extraction (HOG and Colour Histograms) and Scikit-Learn for machine learning classification (SVM) provided a rigorous practical exercise in classical computer vision architecture.

6.2 Key Findings and Lessons Learned

The development process revealed critical insights into the nature of image processing:

- **Memory Constraints Dictate Architecture:** The theoretical design of an algorithm often collides with physical hardware limits. Aggressively downscaling images to 64x64 was a necessary compromise to keep the SVM memory footprint under 100MB, proving that optimization is as important as accuracy.
- **The Fallibility of Color:** While color is an intuitive marker for human categorization, it is a highly volatile metric in computer vision. The infrared night vision challenge perfectly demonstrated how environmental lighting can render color descriptors useless, emphasizing the need for robust structural descriptors like HOG.
- **Classification vs. Detection:** The pipeline operates successfully as an Image Classifier (labeling the whole frame) rather than an Object Detector (drawing bounding boxes). This clarifies the fundamental difference between analyzing a global feature vector versus applying sliding-window localization techniques.

6.3 Future Improvements

While the classical approach provides a deep understanding of feature mathematics, it hits a performance ceiling when dealing with severe background occlusion and extreme lighting variances. Future iterations of this system would benefit immensely from transitioning to Deep Learning frameworks.

7. REFERENCES

7.1 Datasets

- **Caltech Camera Traps (CCT-20):** Database of camera trap images collected from the National Park Service (NPS) and the United States Geological Survey (USGS). Contributed by Sara Beery.

7.2 Software and Libraries

- **OpenCV (Open Source Computer Vision Library):** Utilized extensively for image manipulation, grayscale conversion (`cv2.cvtColor`), Histogram of Oriented Gradients (`cv2.HOGDescriptor`), and color distribution mapping (`cv2.calcHist`).
- **Scikit-Learn (`sklearn`):** The primary machine learning backbone, utilized for data splitting, feature scaling (`StandardScaler`), and multi-class Support Vector Classification (`SVC`).
- **Python 3.10+:** The core programming language used to construct the pipeline.
- **NumPy:** Utilized for highly optimized, multi-dimensional array concatenations and matrix operations.
- **Matplotlib:** Used for rendering performance visualization charts and confusion matrices.
- **Joblib:** Utilized for serializing and deserializing the trained `.pkl` model and scaler files for rapid inference.

7.3 Documentation

- **OpenCV Documentation:** API Reference for HOG algorithm constraints and parameter tuning.
- **Scikit-Learn Documentation:** Reference for Radial Basis Function kernels and Support Vector constraints.

Mini Project GitHub Link: <https://github.com/Sanjay-Krishna001/PCV>